
Tiger Technology Overview



Preliminary June 30, 2004



Apple Computer, Inc.
© 2004 Apple Computer, Inc.
All rights reserved.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, mechanical, electronic, photocopying, recording, or otherwise, without prior written permission of Apple Computer, Inc., with the following exceptions: Any person is hereby authorized to store documentation on a single computer for personal use only and to print copies of documentation for personal use provided that the documentation contains Apple's copyright notice.

The Apple logo is a trademark of Apple Computer, Inc.

Use of the "keyboard" Apple logo (Option-Shift-K) for commercial purposes without the prior written consent of Apple may constitute trademark infringement and unfair competition in violation of federal and state laws.

No licenses, express or implied, are granted with respect to any of the technology described in this document. Apple retains all intellectual property rights associated with the technology described in this document. This document is intended to assist application developers to develop applications only for Apple-labeled or Apple-licensed computers.

Every effort has been made to ensure that the information in this document is accurate. Apple is not responsible for typographical errors.

Apple Computer, Inc.
1 Infinite Loop
Cupertino, CA 95014
408-996-1010

Apple, the Apple logo, AppleScript, Carbon, Cocoa, iCal, iTunes, Mac, Mac OS, Macintosh, Quartz, QuickDraw, and QuickTime are trademarks of Apple Computer, Inc., registered in the United States and other countries.

Gadget, iPhoto, Panther, Rendezvous, Safari, Sound Manager, Velocity Engine, and Xcode are trademarks of Apple Computer, Inc.

Objective-C is a trademark of NeXT Software, Inc.

Adobe, Acrobat, and PostScript are trademarks or registered trademarks of Adobe Systems Incorporated in the U.S. and/or other countries.

Java and all Java-based trademarks are trademarks or registered trademarks of Sun Microsystems, Inc. in the U.S. and other countries.

OpenGL is a trademark of Silicon Graphics, Inc.

Simultaneously published in the United States and Canada.

Even though Apple has reviewed this manual, APPLE MAKES NO WARRANTY OR REPRESENTATION, EITHER EXPRESS OR IMPLIED, WITH RESPECT TO THIS MANUAL, ITS QUALITY, ACCURACY, MERCHANTABILITY, OR FITNESS FOR A PARTICULAR PURPOSE. AS A RESULT, THIS MANUAL IS SOLD "AS IS," AND YOU, THE PURCHASER, ARE ASSUMING THE ENTIRE RISK AS TO ITS QUALITY AND ACCURACY.

IN NO EVENT WILL APPLE BE LIABLE FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES RESULTING FROM ANY DEFECT OR INACCURACY IN THIS MANUAL, even if advised of the possibility of such damages.

THE WARRANTY AND REMEDIES SET FORTH ABOVE ARE EXCLUSIVE AND IN LIEU OF ALL OTHERS, ORAL OR WRITTEN, EXPRESS OR IMPLIED. No Apple dealer, agent, or employee is authorized to make any modification, extension, or addition to this warranty.

Some states do not allow the exclusion or limitation of implied warranties or liability for incidental or consequential damages, so the above limitation or exclusion may not apply to you. This warranty gives you specific legal rights, and you may also have other rights which vary from state to state.

Contents

[Introduction to Tiger Technology Overview](#) 9

[Searching With Metadata](#) 13

[Where Does the Metadata Come From?](#) 13

[Choosing Appropriate Metadata Keys](#) 14

[Running Queries](#) 15

[Core Data](#) 17

[Search Kit Enhancements](#) 19

[SQLite Support](#) 21

[Sync Services](#) 23

[64-Bit Support](#) 25

[Knowing When to Create 64-Bit Binaries](#) 25

[Interoperability With 64-Bit Programs](#) 26

[Dashboard](#) 27

[Pipeline](#) 29

[Using Pipeline](#) 29

[Creating Action Plug-ins](#) 30

[Quartz Composer](#) 33

[Core Image](#) 35

[Core Image](#) 35

[Core Video](#) 36

Resolution Independent UI 37

Scaling Your User Interface 37

Scaling Modes 37

Guidelines for Supporting Scaling Modes 39

Icon Services Changes 39

Core Graphics Support 39

PDF Kit 41

Web Kit and Safari 43

HTML Editing 43

DOM Support 43

Safari Plug-ins 44

QuickTime Enhancements 45

QuickTime Cocoa Framework 45

Video Enhancements 45

Audio Enhancements 46

OpenGL Enhancements 47

OpenAL Support 49

Core Audio Enhancements 51

New Audio Units 51

Audio Unit Test Environment 51

Aggregate Device Support 51

Extended Audio File API 52

Audio File Components 52

Core Audio Clock API 52

Where to Find More Information 52

Universal Access Improvements	53
iChat Presence	55
Fast Logout and Autosave	57
XML Parsing in Cocoa	59
Installer Enhancements	61
Xcode Update	63
Access Control Lists	65
Networking Enhancements	67
Rendezvous Enhancements	67
CFNetwork Enhancements	67
Network Diagnostics	68
KEXT Stable API	69
Document Revision History	71

C O N T E N T S

Figures

Dashboard 27

Figure 1 Dashboard layer in Tiger 27

Pipeline 29

Figure 1 Creating a workflow 30

Quartz Composer 33

Figure 1 Quartz Composer window 34

F I G U R E S

Introduction to Tiger Technology Overview

The forthcoming release of Tiger contains many new technologies, developer-requested improvements, bug fixes, and other enhancements that can affect the way you write software. Some of these features will change the way users work with Mac OS X, and some will affect developers. This document describes many key new features and offers advice on why you might want to take advantage of them and in which situations it would be appropriate to do so.

Important

This is a preliminary document for features in development. Although this document has been reviewed for technical accuracy, it is not final. Apple Computer is supplying this information to help you plan for the adoption of the technologies and programming interfaces described herein. This information is subject to change, and software implemented according to this document should be tested with final operating system software and final documentation. Newer versions of this document may be provided with future seeds of Mac OS X. For information about updates to this and other developer documentation, view the New & Updated sidebars in subsequent seeds of the Reference Library.

Who Should Read This Document

This document is intended for developers who are updating or have plans to update their software to take advantage of Tiger-specific features. This document offers a high-level summary of new features and provides guidance on how you can incorporate them into your software.

Document Organization

This document contains the following articles:

- [“Searching With Metadata”](#) (page 13) describes the system support for metadata.
- [“Core Data”](#) (page 17) describes the new data modeling technology for applications.

- [“Search Kit Enhancements”](#) (page 19) describes improvements to the Search Kit.
- [“SQLite Support”](#) (page 21) describes the built-in database support.
- [“Sync Services”](#) (page 23) describes the support for synchronizing your application data with the system.
- [“64-Bit Support”](#) (page 25) describes tool support for creating 64-bit programs.
- [“Dashboard”](#) (page 27) introduces a new tool for creating lightweight applications.
- [“Pipeline”](#) (page 29) describes a new tool for automating common workflows on your computer.
- [“Quartz Composer”](#) (page 33) describes a new tool for designing visual effects without writing code.
- [“Core Image”](#) (page 35) describes new support for manipulating image and video data.
- [“Resolution Independent UI”](#) (page 37) describes support for scaling graphics to suit user’s needs for more detail or larger content.
- [“PDF Kit”](#) (page 41) describes new support for creating PDF documents.
- [“Web Kit and Safari”](#) (page 43) describes enhancements to the Web Kit frameworks and a new plug-in model for Web Kit–based applications like Safari.
- [“QuickTime Enhancements”](#) (page 45) describes improvements to QuickTime support.
- [“OpenGL Enhancements”](#) (page 47) describes improved support for OpenGL features.
- [“OpenAL Support”](#) (page 49) describes the new support for OpenAL on the system.
- [“Core Audio Enhancements”](#) (page 51) describes improvements to Core Audio support.
- [“Universal Access Improvements”](#) (page 53) describes support for Apple’s new screen reader technology.
- [“iChat Presence”](#) (page 55) describes new support for obtaining information about iChat users.
- [“Fast Logout and Autosave”](#) (page 57) describes support for automatically saving files and reopening them the next time the user logs in to the system.
- [“XML Parsing in Cocoa”](#) (page 59) describes new support for parsing XML data.
- [“Installer Enhancements”](#) (page 61) describes new features of the Mac OS X Installer application.
- [“Xcode Update”](#) (page 63) describes improvements to the Xcode IDE.
- [“Access Control Lists”](#) (page 65) describes support for ACL-based permissions in the file system.
- [“Networking Enhancements”](#) (page 67) describes improvements to Rendezvous and CFNetwork and new network-based diagnostics features.
- [“KEXT Stable API”](#) (page 69) describes new interfaces for writing maintainable kernel extensions.

See Also

You can find downloadable disk images for many WWDC sessions at <http://connect.apple.com/>. Under Download Software you can find areas for each track for WWDC 2004: Application Technologies, Development Tools, Enterprise IT, Graphics and Media, Hardware, OS Foundation, QuickTime and Digital Media, and Sunday Pre-Conference. Look in these areas to find the disk images available for that track. Disk images contain a `Readme.rtf` file pointing to documentation plus things like sample code and demo applications.

Searching With Metadata

Organizing data on a computer is a difficult job, and in most cases it is the sole responsibility of the user. However, even the most organized user may find it nearly impossible to arrange their files in a way that makes it easy to find information. Because the underlying file systems offer only one way of organizing information, users must resort to special tools to search for what they want. Most search tools approach the problem in different ways, so users tend to use multiple products to get what they want. Also, users may want to search more than files. Users may want to search their mail archives, address book contacts, or other digital assets embedded inside a file.

Mac OS X “Tiger” includes a new metadata search technology as a way of finding information in the system. It operates on metadata associated with files and other digital assets and uses that metadata to provide a more robust search experience for users. The metadata can include familiar information such as an asset’s modification date and author but it can also include other information that is custom to a particular asset. For example, an image file might have metadata describing the image’s dimensions and color model. Similarly, a mail message might have metadata describing who sent the message, the subject, and whether the message contained any attachments.

Where Does the Metadata Come From?

For the Tiger metadata search technology to work, it has to have a good set of metadata. Although some bits of metadata (modification dates, display name, pathname) are easy to gather for a given file, most of the interesting data is embedded inside the file. To gather this embedded information, Tiger uses metadata importers.

A metadata **importer** is a small plug-in bundle that you create to extract information from files created by your application. Tiger employs importers to extract metadata from documents on the system. It then stores this metadata in a cache for fast access later. Importers run in the background to gather information about new and existing files.

Apple provides metadata importers for many standard file types that the system uses, including RTF, JPEG, and MP3. However, if you define custom document formats, you must create importers for those formats.

Important

It is imperative that developers provide metadata importers for their own custom document formats. Importers improve the user

experience greatly by making sure your documents can be found during searches.

Writing metadata importers involves parsing your document format for relevant information and adding it to the system data store. When you add a piece of information to the data store, you assign it to an appropriate key. Keys help index the content in the data store and facilitate searches. The Metadata framework (located in the Core Services umbrella framework) defines some standard keys and you can define custom keys as needed to tag your content. When writing importers, it is also important to keep the following guidelines in mind:

- Reuse existing keys whenever possible. Avoid creating new metadata keys if an existing key would be appropriate. Searches work best when developers coordinate on the type of metadata they plan to share for a given type of asset. For example, programs that play MP3 or similar types of files should all use the same keys.
- Use a modified form of the reverse DNS naming conventions as a prefix for any keys that are exclusive to your document types. In the modified form, you need to replace each period character with an underscore character. For example, the Mail program would use `com_apple_mail` as the prefix for all of its keys.
- Incorporate metadata directly into your file formats. Avoid using external files to store metadata content. All critical metadata should be in the same file as the data.
- Let users know what metadata you include in your file formats. Give them a chance to omit that data from files they create. For example, users may not want their user ID or other personal information embedded in files they distribute externally. Consider giving the user an option to save a copy of the file without metadata for external distribution.
- Metadata importers should be able to extract metadata from documents quickly and with minimal effort. Avoid burying metadata deep inside a file, especially if finding that metadata would be computationally intensive later. Define your file format so that relevant information is in the header or in an easily accessible location. Test your importer on large sets of files as well as on very large files.
- Do not expect your application to be running at the same time as your metadata importer. Tiger may call importers at any time to extract metadata from a file. Your importer should be able to extract the information without any assistance from the application that created the file.

For information on how to write an importer, see *Metadata Importers* and *MDImporter Reference* in Carbon File Management Documentation.

Choosing Appropriate Metadata Keys

Choosing appropriate keys for your file's metadata is important for users. Apple defines many standard keys and you should use these keys whenever possible. Using standard keys promotes consistency across different document types and makes the available metadata more valuable to the user. This consistency means that users do not have to distinguish between different attributes that mean essentially the same thing. For example, do not define a key for a file's creator when a key identifying the file's author already exists.

Even though Apple defines many standard keys, there might still be situations where you need to define a new key. You might need a new key to accommodate metadata that is specific to your file type. For example, a graphics program might define a key identifying the color space (CMYK, RGB, sRGB, and so forth) used for an image file. Before defining new keys, however, check to see if other developers have created similar keys for the same or a closely related purpose. If other keys exist, use those keys.

Running Queries

The metadata search technology includes a query API that you can use in your applications to search for files based on their metadata. Using this API, you initiate a query and then iterate the results list to get the returned data. You can sort the results based on different attributes and use those sorting criteria to organize the data before you present it to the user.

Before reading the results of a query, you should always disable updates. Because the results list is updated dynamically as results arrive, if you do not disable updates, the list might change while you are reading data from it. These changes could cause results to be listed multiple times or not at all. Disabling updates temporarily halts the addition of new data to the results list. When you are done, you can then enable updates to continue gathering new results.

For information on how to execute queries, see *MDQuery Reference* in Carbon File Management Documentation.

Core Data

The Core Data framework (`CoreData.framework`) is a new Cocoa technology that integrates with Cocoa bindings to make it even easier to create applications based on a Model-View-Controller architecture. Core Data applies to the data portion of your model and helps you persist your data to any number of different storage mediums. A new tool in Xcode lets you graphically create a schema that describes the data entities in your application. Instances of these entities are then managed by the Core Data framework and stored to a storage medium such as an XML file or SQLite database.

By using Core Data to manage your data model, you significantly reduce the amount of infrastructure code you have to write for your application. You no longer have to worry about the problems associated with object lifecycle management. Core Data handles tasks such as retrieving object data from disk, maintaining references to those objects, and writing modified objects back to disk. Core Data also provides the following features:

- Management of undo/redo beyond basic text editing
- Support for validation of property values
- Support for propagating changes and ensuring that the relationships between objects remain consistent
- Grouping, filtering, and organizing data in memory and transferring those changes to the user interface through Cocoa bindings

When you combine Core Data with Interface Builder and Cocoa bindings, you have a very powerful way to develop advanced applications rapidly. Core Data implements much of the model and controller portions of your application, Interface Builder supports the creation of your views, and Cocoa bindings provide additional controller support to connect the pieces together.

Core Data is intended for applications that have some amount of structured data to manage. Core Data operates on data that has known attributes and defined relationships to other data in your application. For example, you could use it to manage the contents of a spreadsheet or an object-oriented graphics application, such as Sketch; however, you would probably not use it to manage the bits of a bitmap image.

If you are starting to develop a new application, or are planning a significant update to an existing application, you should seriously consider using Core Data. For more information about Core Data, including how to use it in your applications, see *Core Data Concepts* in Cocoa Design Guidelines Documentation.

Search Kit Enhancements

The Search Kit in Mac OS X “Tiger” contains several improvements over the version introduced in Panther. Search performance is now significantly faster than before, with indexing up to three times faster than before. By adopting several new function calls, you can also achieve search-as-you-type performance with incremental results. The Search Kit also includes the following capabilities:

- The ability to search for text contained within a word or at the end of a word
- Optional proximity indexing and quoted phrase searching
- The ability to combine search patterns to form complex search expressions
- Improved relevance ranking with absolute relevance to support incremental results
- Greater control over indexing and searching options
- Improved search syntax
- Unranked search support, which improves performance

If your software does searching but does not yet use the Search Kit, you should investigate the version available in Tiger. The greatly improved performance and new features make the Search Kit much more powerful than previous versions.

For more information, see *Search Kit Reference* in User Experience Carbon Documentation.

SQLite Support

Mac OS X “Tiger” incorporates the SQLite library into the system to provide an embeddable SQL database engine for applications. Programs that link with this library can access SQL databases without running a separate RDBMS process. You can use the SQLite library to create local database files and manage the tables and records in that file. SQLite is optimized to provide fast access to database records and is designed for general-purpose use. For example, Core Data uses SQLite extensively to provide backend storage for user-defined data.

The SQLite library is located at `/usr/lib/libsqlite.dylib`; the `sqlite.h` header file is in `/usr/include`. A command-line interface (`sqlite`) is also available so that you can communicate with SQLite databases using scripts; for details, see the man page for `sqlite`.

For more information about using SQLite, go to <http://www.sqlite.org>.

Sync Services

Sync Services provides data synchronization capabilities to all Mac OS X applications. Third-party applications can use Sync Services to synchronize data with system databases, such as those provided by Address Book and iCal; they can also synchronize custom data with other applications. Sync Services also supports synchronizing data across multiple computers through the user's .Mac account. Whereas the iSync application supported a limited number of devices and applications, Sync Services opens up the world of synchronization to all applications. The iSync application still exists but is now used to initiate the synchronization process for hardware devices that require it.

The Sync Services framework (`SyncServices.framework`) is implemented using Objective-C and is accessible to both Carbon and Cocoa developers. Applications with data to synchronize use the objects in this framework to initiate sync sessions and to push and pull records from the central truth database maintained by the sync engine. Record conflicts are resolved through the sync engine itself, which fixes some conflicts automatically and communicates with the user for most others.

Sync Services is designed for the synchronization of structured data. The system provides predefined schemas for contacts, calendars, and bookmarks in the `/System/Library/SyncServices/Schemas` directory. You can also distribute custom schemas for your own data types and register them with Sync Services.

The synchronization of data takes place using either fast syncing mode or slow syncing mode. The first time your application contacts the sync engine, it must use slow syncing mode to push all of its records to the database and pull the computed changes back. After this initial push, you can then use fast syncing mode, which only pushes and pulls changed records. As needed you can also perform a refresh sync to reload all of the records.

If you have data you want to synchronize with other applications, you should include support for Sync Services in your application design. In addition, you should pay attention to the following guidelines:

- Your user interface should include a way for the user to turn syncing on and off. You should also include whatever windows or controls are necessary to convey the progress of sync operations.
- Maintain state information about which records you have synchronized so that you can use fast syncing mode. This mode makes it possible to sync data more frequently and efficiently than in slow syncing mode.

- Define custom schemas only for new classes of data. Do not redefine an existing schema to more closely match the way your application handles data.
- Be sure to push changes whenever your application quits.

For more information about using Sync Services in your application, see *Sync Services* in Cocoa Interapplication Communication Documentation. For reference information, see *Sync Services Reference for Objective-C* in Cocoa Data Management Documentation.

64-Bit Support

With the introduction of the PowerMac G5, Apple brought 64-bit computing to the desktop of the average user. Initially, developers had to write 64-bit in assembly language, which is not the most convenient language if you have a large code base. The version of Xcode Tools that ships with Mac OS X “Tiger” now provides enhanced support for the compilation, linking, and debugging of 64-bit binaries using C, C++, and Objective-C.

Note: The GCC version 3.3 compiler delivered to WWDC 2004 attendees currently supports 64-bit compilation; however, support for this feature will be removed prior to the release of Tiger to the general public. At that time, a newer version of GCC will be provided to support the creation of 64-bit binaries. You will need to recompile your apps before releasing them for use with Tiger.

Knowing When to Create 64-Bit Binaries

The vast majority of developers do not need to migrate their applications to a 64-bit model. A 64-bit application does not necessarily perform better than its 32-bit equivalent. In fact, a 64-bit binary has a larger memory footprint because all of its pointers are double the size of those in 32-bit binaries. However, there are situations when compiling code into a 64-bit binary is important. These situations include the following:

- An application allocates contiguous data blocks of more than 2 gigabytes in size.
- An application requires more than 2 gigabytes of data in memory on a regular basis.

The types of applications that usually require 64-bit support are scientific applications and other applications that handle large amounts of data. For example, complex simulations typically operate on large data sets and might need this support. Also, a 3D rendering application might need 64-bit support to handle increasingly complex models.

Even if your application works on large amounts of data, there are situations where compiling it into a 64-bit binary might not be appropriate. For example, suppose you have a graphics program that operates on high-resolution images. Given that only a portion of the image can be viewed at once, it might be better to load just the portion you need into memory. Similarly, a multimedia application might operate on a large amount of data but only in small chunks at any given time.

All libraries in a 64-bit program must be compiled as 64-bit libraries. For Tiger, only the `libsystem` shared library offers 64-bit support. Mac OS X does not support a mixed mode for creating binaries with both 32-bit and 64-bit code. If you are creating an application that does need to be a 64-bit binary, you should consider using a client-server model that separates your data management code from any user interface code. In this scenario, the server would be your 64-bit application running as a faceless BSD process and your client would be your GUI-based application.

Interoperability With 64-Bit Programs

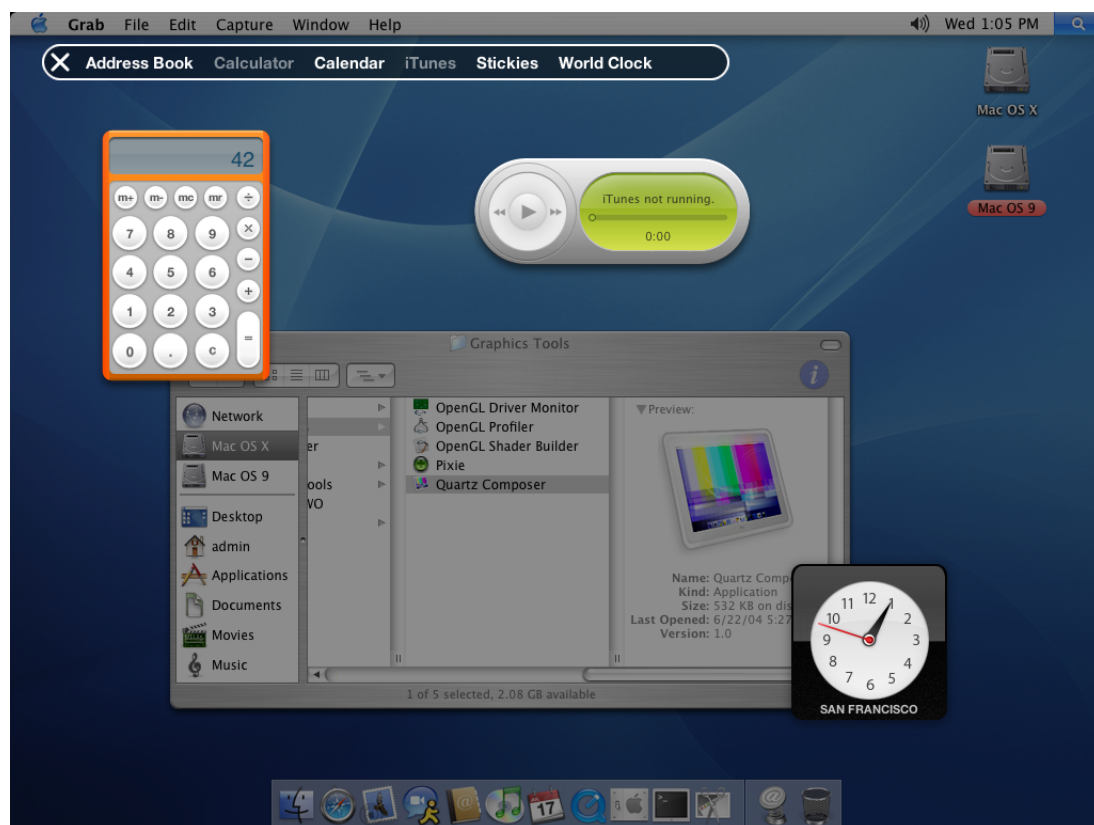
Even if your application can remain a 32-bit binary, you need to be aware that other programs may be either 32-bit or 64-bit binaries. If you exchange data with other applications, you must be explicit about the data you pass around. All data types you use should have explicit sizes associated with them. For example, rather than using a generic `int` data type for a 32-bit value, you should use the `uint32_t` data type.

For more guidelines on 64-bit support, see *64-Bit Transition Guide* in Darwin Porting Documentation. You can also refer to the section “Porting Tips” in *Apple Software Design Guidelines* in Mac OS X Documentation.

Dashboard

Dashboard is a new feature in Mac OS X “Tiger” that you can use to implement lightweight desktop utilities. Dashboard lets you create small, specialized bundles (called **gadgets**) that operate within a special layer of the Mac OS X desktop. When activated by the user, this layer comes to the foreground and provides access to the currently selected gadgets. Dashboard ships with several standard gadgets, including a calculator, clock, and iTunes controller, as shown in Figure 1.

Figure 1 Dashboard layer in Tiger



Creating gadgets is simpler than most applications because gadgets are effectively HTML-based applications. You create your gadget interface using HTML and display it in the gadget window, which embeds a web view. You can include JavaScript code in your gadget to provide dynamic behavior. Dashboard also includes some Web Kit extensions to provide advanced features, such as complex rendering using Quartz-like JavaScript calls. For features that require your own custom code, Dashboard includes an interface for creating native plug-ins that you can access from your gadget using JavaScript.

Important

The interfaces for the Dashboard JavaScript objects are preliminary and are very likely to change prior to the release of Mac OS X version 10.4. Developers should not plan on shipping gadgets developed with the WWDC release of the technology.

If you have program ideas for which it would be overkill to create an application, you might consider creating a gadget instead. Gadgets are intended to be lightweight entities that display information a user might glance at occasionally. You can use them to display dynamic web content, such as stock quotes or package tracking information, or to expose features of a frequently used application.

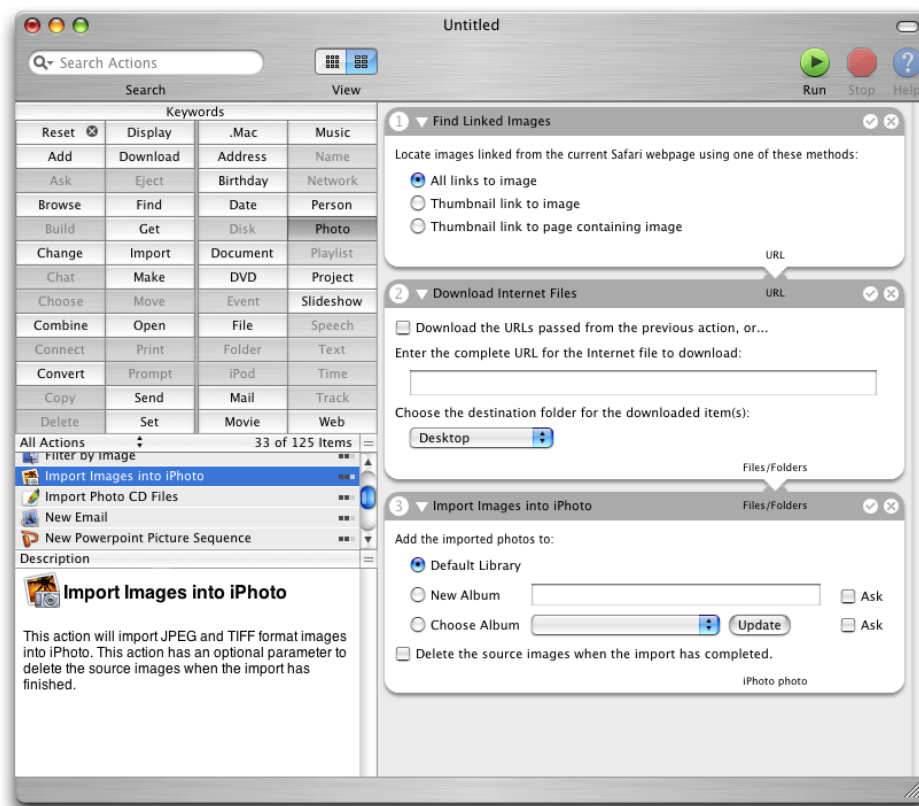
For information on how to create a gadget, see *Dashboard Tutorial* in Apple Applications Documentation. For reference information about the JavaScript objects and plug-in interfaces used by gadgets, see *Dashboard Developer Reference* in Apple Applications Documentation.

Pipeline

Pipeline is a new tool that lets you automate common procedures on your computer. Both developers and end users can create complex workflows quickly, and without writing any code. These workflows can take advantage of many features from Mac OS X “Tiger” and many standard applications such as Mail and iPhoto. Developers can also make the features of their own applications available to Pipeline users or define new code modules to provide additional features.

Using Pipeline

Pipeline is based on the concept of a workflow, which is simply a procedure that you want to automate. A workflow comprises one or more building blocks, called “actions.” Actions represent tangible tasks—opening a file, saving a file, applying a filter, and so on—that can be performed either by another application or by a standalone code module. Using the Pipeline interface, you create a workflow by organizing the desired actions in a way that lets data flows from one action to the next. Figure 1 shows an example of a workflow that extracts all linked images from the currently open Safari webpage and adds them to the user’s iPhoto library.

Figure 1 Creating a workflow

Once you have a workflow, you can run it directly from the Pipeline interface or save it as a separate application.

Note: If you save a workflow as an application, you can run that application only on Tiger-based systems.

Creating Action Plug-ins

In addition to creating workflows, developers can also define new actions for use in Pipeline. Actions are implemented through action plug-ins, which you create using Xcode. Each plug-in contains a nib file and some code to manage the action's user interface and implement its intended behavior. You write the code for your plug-in using either AppleScript or Objective-C.

When designing action plug-ins, remember to keep their behavior as concise as possible. Actions are intended to be building blocks, so they should represent a single task. If your actions incorporate too many features, they may become too specialized for general use. Thus, it is preferable to create multiple action plug-ins to represent individual tasks than to create one plug-in that combines the behaviors of those tasks into a more complex task.

If you are developing an application, you should think about providing action plug-ins for your application's most common tasks. Providing these plug-ins gives users a good starting point for using your application with Pipeline. AppleScript is one of the easiest ways for both developers and users to create action plug-ins because it takes advantage of an existing interface for controlling your application. When you ship your application, you should consider shipping an initial set of actions based on your AppleScript terminology.

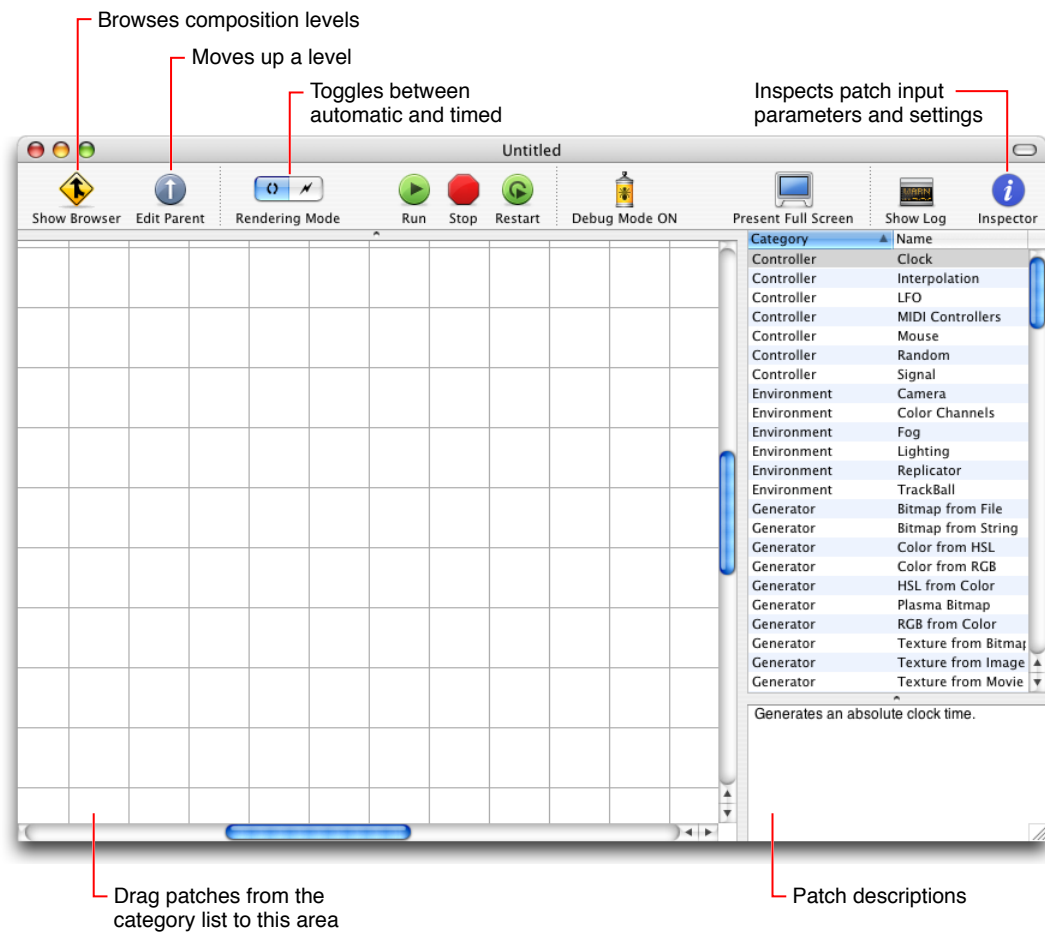
Pipeline is not limited to AppleScript-based actions. You can use the Pipeline Engine framework (`PipelineEngine.framework`) to create an action that is also a standalone code module. This framework defines an Objective-C interface for creating an action plug-in. From your plug-in, you can call most other Mac OS X APIs to communicate with the system or with other processes. For example, you could write an action plug-in to draw an image directly to the screen using Core Graphics or to communicate with another process using sockets.

Quartz Composer

Quartz Composer is a development tool provided with Mac OS X “Tiger” for processing and rendering graphical data. Quartz Composer provides a visual development environment built on technologies such as Quartz 2D, Core Image, OpenGL, and QuickTime. You can use Quartz Composer as an exploratory tool to learn the tasks common to each visual technology without having to learn its application programming interface (API). In addition to supporting visual technologies, Quartz Composer also supports nongraphical technologies such as MIDI System Services and Rich Site Summary (RSS) file content.

Quartz Composer creates compositions, which are graphical programs that run autonomously. Figure 1 shows a document window for Quartz Composer. Using this window and the available tools, you assemble the graphic elements you need and patch them together to create your composition.

Figure 1 Quartz Composer window



Developers can use the QCView and QCRenderer Objective-C classes to play back and interact with compositions from within an application. Any compositions you create are fully compliant with the Cocoa controller layer and key-value bindings, allowing you to combine Interface Builder and Quartz Composer to rapidly develop data-driven visualizations.

For information on how to use Quartz composer, see *Visual Computing With Quartz Composer* in Graphics & Imaging Quartz Documentation.

Core Image

Mac OS X “Tiger” introduces two new media technologies: Core Image and Core Video. These technologies allow developers to tap into the power of the GPU and achieve unprecedented speed and performance through an easy-to-use architecture.

Core Image

Core Image uses GPU-based acceleration and 32-bit floating point support to provide fast image processing and pixel-level accurate content. The plug-in based architecture lets you expand the capabilities of Core Image through the creation of Image Units, which implement the desired visual effects.

Image Units apply visual effects using filter objects. A filter object imparts a set of attributes to an image or sequence of images. The process for creating and applying a filter object is the same regardless of the Image Unit type. What changes are the attributes you set for the object. For example, to perform a white-point adjustment, you specify your image and the color value that represents the new white point; to perform a dissolve transition, you specify the source image, destination image, and a transition time.

Core Image includes built-in Image Units to perform the following tasks:

- Crop images
- Correct color, including perform white-point adjustments
- Apply color effects, such as sepia tone
- Blur or sharpen images
- Composite images
- Warp the geometry of an image by applying an affine transform or a displacement effect
- Generate color, checkerboard patterns, Gaussian gradients, and other pattern images
- Add transition effects to images or video
- Provide real-time control, such as color adjustment and support for sports, vivid, and other video modes
- Apply linear lighting effects, such as spotlight effects

In addition to the built-in Image Units, you can also define custom Image Units using the classes of the Core Image framework. You can use both the built-in and custom Image Units in your application to implement special effects and perform other types of image manipulations. Image Units take full advantage of Velocity Engine, Quartz, OpenGL, and QuickTime to optimize the processing of video and image data. Rasterization of the data is ultimately handled by OpenGL, which takes advantage of graphics hardware acceleration whenever it is available.

Core Image is part of the Quartz Core framework (`QuartzCore.framework`). For information about how to use Core Image or how to write custom Image Units, see *Image Processing With the Core Image Framework* and *Core Image Reference* in Graphics & Imaging Quartz Documentation.

Core Video

Core Video provides a modern foundation for video services in Mac OS X “Tiger.” It delivers a bridge between QuickTime and the GPU for hardware-accelerated video processing. This highly optimized pipeline increases performance and reduces CPU load, freeing up resources for other operations. Core Video also allows developers to apply all the benefits of Core image to video: blazingly-fast performance of filters and effects, per-pixel accuracy, and hardware scalability.

Core Video is part of the Quartz Core framework (`QuartzCore.framework`).

Resolution Independent UI

Mac OS X “Tiger” introduces support for “resolution independent UI”, which enables users to choose between viewing more detail or a larger user interface for the same screen resolution. Resolution independent UI will have an effect on the way you lay out your user interface. It will also affect the graphics you create for your application. The following sections provide an overview of the changes you need to make to your applications to support this feature.

Scaling Your User Interface

In the past, many frameworks assumed that content was displayed at 72 dpi. As a result, the applications that used those frameworks operated with the same assumption. With resolution independent UI, the rules have changed. Now, applications need to be prepared to scale the contents of windows based on the user’s preferences. Although many system frameworks handle scaling for you automatically, you may still need to make changes to your applications to support the feature.

When a scaling factor is in effect, the system provides this factor to applications as a multiplier of the base resolution of 72 dpi. Thus, if the user chooses to display content at 72 dpi, the scaling factor is 1.0. If the user chooses to display content with greater detail, the scaling factor may be higher than 1.0. When designing your application, you can continue to make initial measurements based on a 72-dpi resolution. However, depending on the technologies you are using, you must then choose an appropriate scaling mode to determine how the scaling factor will be applied.

Note: Users will not be able to set scaling factors in Tiger; only developers will be able to set them using Quartz Debug or the `defaults` command. Eventually, scaling factors will be a user-controlled setting.

Scaling Modes

Tiger defines three scaling modes for rendering content: **framework-scaling**, **app-scaling**, and **magnified**. Each scaling mode offers a certain amount of support for applying scaling factors to rendered graphics. If you are using modern technologies, such as Cocoa and HView, much of this support is automatic, but if you use older technologies, such as QuickDraw, you may have to do some of the work required to scale your graphics.

Framework-Scaling Mode

Framework-scaling mode is named for the fact that most scaling is handled automatically by modern graphics systems such as the Application Kit or HView. Framework-scaling mode is the preferred mode for most content and is the only mode available to Cocoa applications. If you are writing a Carbon application, you must use HView and composited windows to take advantage of framework-scaling mode.

In framework-scaling mode, the size of the window buffer is increased to accommodate the actual number of pixels that are going to be drawn on the screen. During drawing, the system frameworks automatically scale controls and other interface elements, such as menus and the window title bar, to the appropriate sizes. The system frameworks also add a scaling transform to the graphics context (CGContext) you use to render content in the window. Any calls you make to Quartz or the Application Kit are thereby scaled automatically.

Framework-scaling mode is not appropriate for applications that use QuickDraw calls. The scaling factors applied to the graphics context will have no effect on your QuickDraw content; therefore, any content you draw with QuickDraw will be unscaled. Applications that use QuickDraw should use the app-scaling mode instead.

Note: Because Cocoa applications support only framework-scaling mode, you should make sure your Cocoa code does not contain QuickDraw calls. If it does, you should replace them with equivalent calls to Quartz so that you can use framework-scaling mode.

App-Scaling Mode

App-scaling mode is so named because the application is responsible for handling all scaling of content. In this mode, the size of the window buffer is increased to accommodate the actual number of pixels that are going to be drawn on the screen. The system continues to scale system-defined controls, including the window title bar and menus, but it does not apply a scaling transform to the window's graphics context. Instead, your application assumes full responsibility for scaling all of its window content to the correct size.

App-scaling mode is intended for developers who still use legacy technologies such as QuickDraw or for developers who prefer to handle scaling factors themselves. To use this mode, you must query the system for the current scaling factor and use it to create an appropriate transform, which you can then apply to your content.

Magnified Mode

Magnified mode is a compatibility mode to provide basic support for scaling until you can modify your applications to use one of the other modes. In magnified mode, the window server applies the current scaling factor to the window buffer to create a magnified view of the image. Because it is based on the original window buffer, the scaled window does not have any more detail than it had originally. Pixels are simply interpolated to fit the new window size. As a result, windows might look like they had simply been magnified by a lens.

Applications do not have to do anything to use magnified mode. It is used by default if you do not specify any other mode when you create your windows. However, magnified mode is not recommended for general usage. The graphics it produces are imprecise and may look ugly when scaling factors above 1.0 are in effect.

Guidelines for Supporting Scaling Modes

The following guidelines offer advice for supporting resolution independent UI in your programs.

- Currently, scaling factors do not change dynamically during the course of a user session. Therefore, your program does not need to check for changes to the current scaling factor.
- If your code calculates the size or position of content based on the size of the parent window or view, your resizing code might need to take scaling factors into account and make corrections to those values as necessary. For example, you might want to adjust the positions of your controls to ensure that they are aligned correctly with each other.
- If your drawing code assumes a fixed-size coordinate system for a view, you might need to take scaling factors into account before invalidating that view or its contents.
- If you expect elements to be a specific size, you might want to ignore scaling factors altogether. For example, you might want a separator line to always be one pixel high, regardless of the scaling factor.
- If your window layout includes informational graphics, such as images or icons, you should prepare those graphics for display at higher resolutions. You can do that using the following techniques:
 - Store line art and nonphotographic content as PDF files instead of as TIFF files.
 - Use 300-dpi images instead of the usual 72 dpi used for photographic content.
 - Use TIFF files with multiple versions of the same image—for example, one at 72 dpi and one at 300 dpi.

For additional information and guidelines on how to support resolution independent UI, see the Carbon and Cocoa release notes in Release Notes Documentation.

Icon Services Changes

To support resolution independent UI, Icon Services in Tiger supports icons that are 256 x 256 pixel in size. To support these larger icons, a new icon type selector has been added for you to use in calls to `SetIconFamilyData` and `GetIconFamilyData`. The selector is `kIconServices256PixelFormatARGB` and is defined in the `IconStorage.h` header file in the Operating System Services subframework (`OSServices.framework`) of Core Services.

Note: The bitmaps you specify for your 256 x 256 pixel icons should not be premultiplied.

Core Graphics Support

Quartz was designed with resolution independence in mind. Since the introduction of Mac OS X, developers have been able to use the same code to render both to a printer or the screen. When it comes to support for resolution independent UI, Quartz in combination with `HIView` or `NSView`

provides most of the support you need automatically. Both `HIView` and `NSView` attach an affine transform matrix to the local drawing context to account for scaling factors. This transform automatically converts coordinates in your views to scaled coordinates.

Even with all the automatic support, there may still be times when you want to know how to adjust your content in advance. For this, Core Graphics provides a new function to get the affine transform used for mapping between your original 72-dpi coordinate space and the scaled coordinate space. This function is defined as follows:

```
CGAffineTransform CGContextGetUserSpaceToDeviceSpaceTransform(CGContextRef  
c);
```

You can use the affine transform returned by this function to make adjustments to the coordinate values of points and rectangles. You do this by applying the transform to a coordinate in your user space to get a new value in the scaled coordinate space. You can then make any necessary adjustments to the scaled coordinate to ensure a proper appearance. For example, you might round off fractional components of scaled coordinates to make sure elements line up properly. You then apply the inverse transformation to the scaled coordinates to get the new coordinates in your user space.

PDF Kit

The PDF Kit is a new Cocoa framework for managing and displaying PDF files directly from the windows and dialogs of your application. Using the classes of the PDF Kit, you can embed a PDFView in your window and give it a PDF file to display. The PDFView class handles the rendering of the PDF content and provides controls for navigation and setting the zoom level. Users can also copy selected text in a PDFView to the pasteboard.

Note: Although it is written in Objective-C, you can also use the classes of the PDF Kit in both Carbon and Cocoa applications. For information on how to do this, see *Integrating Carbon and Cocoa in Your Application* in Cocoa Design Guidelines Documentation.

The PDF Kit classes simplify the work needed to manage PDF content. Most developers need only to embed a PDFView object into their window to display PDF files. The PDFView class handles all of the rendering and management of the displayed PDF data. If you want more control over the data, the framework includes several other classes as well. You can use these classes to get the number of pages in a PDF file, find text, manage selections, add annotations, and specify the behavior of some graphical elements, among other actions.

If you need to display PDF data directly from your application, the PDF Kit is highly recommended. It hides many of the intricacies of the Adobe PDF specification and provides standard PDF viewing controls automatically.

The PDF Kit is part of the Quartz framework (`Quartz.framework`) in Mac OS X “Tiger.” For more information about the PDF Kit, see *PDF Kit Reference* in Cocoa Graphics & Imaging Documentation.

Web Kit and Safari

Mac OS X “Tiger” includes some significant improvements to the Web Kit that give your software more control over HTML content. These improvements include programmatic support for editing HTML content and for accessing and modifying the DOM. Web Kit also defines new interfaces for creating plug-ins that can be used by Safari and other applications that use Web Kit.

HTML Editing

The HTML editing feature of Web Kit enables you to create editable text views containing HTML. The support is equivalent to the RTF-based editing support provided by the `NSTextView` class in Cocoa but uses the Web Kit to manage the HTML. You can use this support to replace text and manipulate the document text and attributes, including CSS properties.

The editing support provided by Web Kit is not intended to provide the features available from professional HTML editing applications. Instead, it is aimed at developers who need to display and handle the basic editing of HTML content.

For information about the new methods for editing HTML with Web Kit, see the *Web Kit Reference for Objective-C* in Cocoa User Experience Documentation. For general information about using the Web Kit in your application, see *Displaying Web Content* in Cocoa Internet & Web Documentation.

DOM Support

Web Kit now includes support for creating and editing content at the DOM level of an HTML document. With the DOM framework, you can navigate DOM nodes and manipulate those nodes and their attributes. You can also use the framework to extract DOM information. For example, you could extract the list of links on a page, modify them, and replace them prior to displaying the document in a web view.

For information about the DOM interfaces, see the header files in `WebKit.framework`.

Safari Plug-ins

In Tiger, Safari supports a new plug-in model for tying in additional types of content to the web browser. This new model is based on an Objective-C interface and offers significant flexibility to plug-in developers. In particular, the new model lets plug-in developers take advantage of the Tiger API for modifying DOM objects in an HTML page. It also offers hooks so that JavaScript code can interact with the plug-in at runtime.

Safari plug-in support is implemented through the new `WebPlugIn` and related objects defined in Web Kit. For information about these objects, see the *Web Kit Reference for Objective-C* in Cocoa User Experience Documentation.

QuickTime Enhancements

Mac OS X “Tiger” includes a new version of QuickTime (version 6.6) that adds enhancements to the audio and video handling capabilities of QuickTime. Tiger also includes a new framework for using QuickTime in Cocoa.

QuickTime Cocoa Framework

Tiger includes a new QuickTime Objective-C framework that provides a rich API for manipulating time-based media. The QuickTime Kit (`QTKit.framework`) allows for movie playback, editing, and export to standard media formats, among other capabilities. The classes in this framework open up a tremendous amount of QuickTime behavior to Cocoa developers. Instead of learning how to use the more than 2500 functions in QuickTime, developers can now use a handful of classes to implement the features they need.

If you are a Cocoa developer and currently using `NSMovie` or `NSMovieView`, you should update your code to use the newer `QTMovie` and `QTMovieView` classes instead. These newer classes offer all of the same features as their older Cocoa counterparts and also offer better manipulation of the movie contents and better in-place editing capabilities.

For an introduction and tutorial on how to use the QuickTime Kit, see *QuickTime Kit Programming Guide* in QuickTime Documentation. For reference information about the QuickTime Kit classes, see *QuickTime Kit Reference for Objective-C* in QuickTime API Reference Documentation.

Video Enhancements

Video for QuickTime 6.6 supports the following improvements and features:

- Support for the H.264 codec.
- B-frame video compression, which enables the following advances:
 - Independent decode and display times for video frames
 - Improved display, editing, and capture in MPEG-2, MPEG-4, and other advanced-codec video formats

- A new visual context, which provides an abstraction layer that goes beyond graphics worlds (GWorlds) and supports rendering using engines other than QuickDraw.
- Support for rendering content using OpenGL
- Support for getting metadata from media files

For more information, see *What's New in QuickTime 6.6 Developer Preview* in QuickTime Documentation.

Audio Enhancements

Audio for QuickTime 6.6 breaks free of the limitations of the Sound Manager, adding a new audio media handler and sound description file format. These changes also make QuickTime compatible with Core Audio devices and data types. Among the improvements and features added to this version are the following:

- Support for sample rates greater than 64 kHz
- Support for multichannel audio, including support for different spatial channel layouts and discrete channels
- Support for sample sizes greater than 16 bits
- Support for playback, editing, capture, import, and export of audio using the enhanced sample rates, sample sizes, and channel support
- Interfaces for accessing raw PCM audio samples from VBR-compressed audio sources
- Preservation of clock information for sample-accurate synchronization
- Support for getting metadata from audio files

Developers using the high-level QuickTime interfaces gain much of this support automatically. If you are using the low-level QuickTime interfaces, you can use several new function calls to gain access to this enhanced support.

For more information, see *What's New in QuickTime 6.6 Developer Preview* in QuickTime Documentation.

OpenGL Enhancements

Support for OpenGL in Mac OS X “Tiger” now includes better support for pixel buffers. Pixel buffers (also known as Pbuffers) let you do offscreen rendering with OpenGL. You can create a pixel buffer once and access it as needed when rendering other objects. For example, you might create a pixel buffer to store a texture that you want to apply to several different objects. OpenGL functions help you create, manage, and destroy pixel buffers.

For more information about using OpenGL in your programs, see *OpenGL for Mac OS in Graphics & Imaging OpenGL Documentation*.

OpenAL Support

Mac OS X “Tiger” adds support Open Audio Library (OpenAL), a cross-platform standard for delivering 3D audio in applications. OpenAL lets you implement high-performance positional audio in games and other programs that require high-quality audio output. Because it is a cross-platform standard, the applications you write using OpenAL on Mac OS X can be ported to run on many other platforms.

Apple’s implementation of OpenAL is based on Core Audio, so it delivers high quality sound and performance on Mac OS X systems. To use OpenAL in a Mac OS X application, include the OpenAL framework (`OpenAL.framework`) in your Xcode project. This framework includes header files whose contents conform to the OpenAL specification, which is described at <http://www.openal.org>.

Core Audio Enhancements

Core Audio in Mac OS X “Tiger” includes new tools and functions to improve the audio handling experience of applications.

New Audio Units

Core Audio includes two new audio units that you can use in your programs:

- A file-playback audio unit lets you use an existing sound file as an input source. The audio unit converts file data from any format to linear PCM so that it can be processed by other audio units.
- A time and pitch transformation audio unit lets you modify both the pitch and time of audio data. You can use the time-shifting features of this audio unit to change the playback speed without changing the pitch. Similarly, you can use the pitch-shifting features to alter the pitches without affecting the playback speed. You can also apply both effects.

Audio Unit Test Environment

Tiger includes the new AU Lab tool, which lets you graphically host audio units and examine the results. AU Lab hosts audio units and lets you use them to do live mixing and playback of audio content. Parameters for the audio units are controlled graphically using a custom interface you design or using a generic interface derived from the audio unit definition.

Aggregate Device Support

Core Audio now supports the creation of aggregate devices. As the name suggests, an aggregate device combines multiple audio devices together under the auspices of a single audio device. For example, if you have two devices with two input channels each, you can use them to create a single aggregate device with four input channels. Core Audio takes care of routing the audio data to the appropriate hardware device.

Extended Audio File API

The extended audio file API (`ExtendedAudioFile.h` in the Audio Toolbox framework) removes much of the complexity previously associated with converting files from one format to another. This API builds on top of the existing Audio File and Audio Converter units to manage the conversion of file data to and from linear PCM format.

Using this API, you can read data in any format supported by Core Audio. The audio data is converted to linear PCM and returned to your application for processing. When you want to write the data back out to disk, you can reverse the process and write out to the same or a different file format, depending on your needs. At all times, your application handles the data in linear PCM format. Core Audio handles the conversion of data based on the target file format.

Audio File Components

Core Audio introduces a new extension mechanism for supporting audio file formats. You can now create components for reading and writing audio file formats not natively supported by Core Audio. When an application attempts to open a nonnative audio file format, the Audio File API checks the registered components to see if one handles the unknown format. If it finds a component, it uses that component to open the file and return data to the client application.

The process for creating an audio file component is similar to the one for creating an audio converter. See the Core Audio SDK on the Development Kits page (<http://developer.apple.com/sdk/>) for more information.

Core Audio Clock API

Core Audio introduces a new clock API for creating and tracking MIDI time formats. The API supports the use of the MIDI Time Code (MTC) and MIDI Beat Clock protocols to synchronize operations between MIDI-based controllers. The header file `CoreAudioClock.h` in the Audio Toolbox framework describes the interfaces for this API.

Where to Find More Information

For detailed information about the new function calls, see the header files of the assorted Core Audio frameworks.

Universal Access Improvements

Mac OS X already includes numerous technologies for making software accessible to all users. Features such as full keyboard navigation, enhanced contrast, sticky keys, screen zooming, and several others all help to improve access for persons with disabilities. Now, Mac OS X “Tiger” extends the legendary Macintosh ease-of-use with a spoken interface for users with visual impairments.

A spoken interface is a way of providing audible cues and keyboard navigation to help visually-impaired users access the Macintosh. With this feature, users navigate the desktop using keyboard commands and receive feedback through a synthesized voice. The synthesized voice describes the objects they are selecting and manipulating with the keyboard, including menus, windows, controls, and text. This feature takes advantage of the existing Accessibility API and uses the information there to describe user interface elements.

As a developer, supporting accessibility is not just a good selling point, it’s also required if you sell your software into educational or government markets. Section 508 mandates that programs be accessible to persons with many types of disabilities. Although Mac OS X provides the bulk of this support, there is still work you need to do. The most important thing you can do is support the Accessibility API in your applications. You should also do the following:

- Provide good descriptions for your user interface elements.
- Identify title elements and link them to their corresponding data entry controls. For example, a label is a title element for a text field control.
- Provide links between related user interface elements. Links make it easy to navigate between related items that may be located in different parts of your window.
- Make sure users can navigate your application using only the keyboard. For example, provide alternatives to drag operations and think of keyboard alternatives for performing actions.
- Do not override reserved keyboard shortcuts.

For more information on supporting accessibility in your applications, see *Getting Started with Accessibility* in Accessibility Documentation.

iChat Presence

Mac OS X “Tiger” introduces support for detecting and displaying the presence of users through iChat. The Instant Message framework lets you find out the current status of a user connected to an instant messaging service. This framework can return a user’s custom icon, a user’s status message, or a URL to an image that indicates a user’s status. For example, a mail program could use the framework to determine if an email is from a user who is currently online; it can then display an appropriate icon next to that person’s name to indicate the user’s availability.

For more information, see *iChat Instant Message Reference* in Apple Applications Reference Documentation.

Fast Logout and Autosave

Fast logout and autosave is a performance-related feature designed to improve the user experience of Mac OS X “Tiger.” Closing windows is an expensive operation, especially if you have to prompt the user about unsaved changes for each document window. This feature makes the logout, shutdown, and restart operations simpler by reducing the number of confirmation dialogs posed to the user while simultaneously protecting the user’s unsaved data.

During a logout, shutdown, or restart operation, the system now adds a parameter to the standard `kAEQuitApplication` Apple event that indicates whether an application should post a confirmation dialog for unsaved changes. If the parameter is present, the application should save all uncommitted changes to a special location along with some state information indicating what was saved. (The application should not overwrite the original document.) The next time the user logs in, the system relaunches each application with autosaved documents and tells them to reconstitute those documents as they were immediately before the last logout. Users may disable the relaunching of applications by holding down the Shift key at login.

Important

If your application cannot safely quit without prompting the user for information, you must prompt the user for that information. Your application should never allow the user’s data to be lost.

Fast logout and autosave should be supported by all applications for which it makes sense. For example, a document-based application should nearly always support this feature to save the user’s documents but a chat program should probably not relaunch a chat session upon startup. Cocoa provides automatic support for the feature through the `NSDocument` and `NSDocumentController` classes. Carbon applications must provide explicit support for this feature.

For information on how to support this feature in your application, see *Fast Logout and Autosave* in Carbon Design Guidelines Documentation.

XML Parsing in Cocoa

A new feature in Mac OS X “Tiger” is the NSXML class, which offers advanced support for manipulating XML-based documents in Cocoa. You can use this class to do the following:

- Create new XML documents
- Search the contents of an XML document arbitrarily
- Convert structured documents using Extensible Stylesheet Language Transformation (XSLT)
- Validate XML content
- Create and manipulate document type definitions (DTDs) as logical tree structures

The NSXML class maintains an internal tree structure to represent XML data. This approach is different from the one used by NSXMLParser, which reads existing XML and returns one node at a time. The presence of the internal tree structure makes it easier to navigate the contents of an XML document using NSXML. It also means you can build an XML document in memory before writing it to a file.

NSXML supports data transformations using XSLT, which let you convert the data from one structured content to another. For example, you can use the XSLT support to convert an existing XML document to XHTML for display in a web browser.

In addition to creating documents, you can also use NSXML to search documents using the XQuery or XPath query languages. These languages let you create complex search expressions to retrieve single nodes, groups of nodes, or atomic values (strings, integers, dates, and so on) from the XML document.

NSXML is not the best solution for all situations where XML must be processed. The internal tree representations can take up a lot of application memory, especially for operations such as validation and XSLT transformations. If you simply need to find certain values in an XML document and do not need a persistent representation of the XML to modify, then NSXMLParser offers better performance.

For more information, see *Tree-Based XML Processing* in Cocoa Data Management Documentation.

Installer Enhancements

The Mac OS X "Tiger" Installer application supports several new features. These features help you deliver a more robust installation experience for your users while ensuring that your software is installed correctly. Among the basic features added to the Installer are the following:

- File version checking. This feature extends the current bundle-version checking mechanism to support checks for individual files. This mechanism can ensure that you only replace outdated files on the user's machine, leaving newer files alone.
- Multiple CD/DVD support. The Installer now supports installation from multiple CDs or DVDs and prompts the user for new discs as they are needed.
- Minimal interface installations. Use this feature to implement automated installs or to reduce the number of panels shown to the user during an installation.
- Application upgrades. If you create an upgrade Installer, you can now let the user directly select the application to be upgraded. Previously, users could only select installation volumes.

In addition to the basic features are two more advanced features: Installer plug-ins and distribution scripts.

Installer plug-ins are bundles that you use to display custom panes in the Installer application. You can use these panes to display instructions or gather additional information, such as registration information. Installer plug-ins are written in Objective-C and take advantage of the Installer Plug-ins framework (`InstallerPlugins.framework`) that ships with Tiger.

Distribution scripts offer a flexible and convenient way to install multiple subpackages from a single installation package. A script itself is a simple collection of XML elements and optional JavaScript code, but you can use it to provide a very sophisticated set of installation options. Using a distribution script, you can establish package dependencies, offer highly configurable installation options, and provide code to prevent the user from installing software on an unsupported hardware or software configuration.

For additional information on Installer-plug-ins and distribution scripts, download the disk image for WWDC session 403, "Using Mac OS X Installer's Features to Deliver Your Application" from the members-only section of <http://connect.apple.com>.

Xcode Update

The Mac OS X "Tiger" Developer Preview includes a preview of Xcode 2.0, Apple's integrated development environment. The Xcode 2.0 preview includes several new features intended to speed up the development process and let you concentrate on designing and creating your software. Many of these features are also available in the Xcode 1.5 preview, which will be shipping prior to the release of Mac OS X version 10.4.

The following features are available only in the Xcode 2.0 preview for Tiger:

- **Workspaces.** Xcode 2.0 introduces different project window configurations, called workspaces, that let you choose the layout that best suits your preferred workflow. The Xcode preview currently ships with three workspaces:
 - The default (or traditional) Xcode project window, introduced in Xcode 1.0.
 - An all-in-one project window that combines views for project management, building, and debugging in a single window.
 - A condensed project window workspace that provides a smaller and more compact project window.
- **Visual design.** Xcode 2.0 includes a visual designer that lets you create class models for C++, Objective-C, and Java classes and persistence models for use with the Core Data framework.

The following features are available in Xcode 2.0 and in the Xcode 1.5 preview:

- **Dead code stripping.** The static linker (`ld`) that ships with the Xcode Tools adds support for stripping unused symbols and data blocks from executable files. This process (known as dead-code stripping) helps reduce the overall size of executables, which in turn improves performance by reducing the memory footprint of the executable.
- **Code completion for Java.** Xcode's code completion interface, which assists you in writing source code by suggesting—and optionally inserting—identifiers and symbol names appropriate for the context that you are currently working in, now supports Java.
- **Faster indexing.** Xcode now performs indexing of your project on a background thread. The project index is used to support features such as code completion and the class browser.
- **Improved editor performance.** Xcode's source editor is speedier and more responsive when opening and editing files.
- **Graphical remote debugging.** You can now debug a program running on a remote host from within Xcode's graphical debugger interface. Users can step through code, view variables,

and manage breakpoints from within Xcode's graphical user interface instead of from the command-line. This feature is useful for debugging programs, such as full-screen applications, that cannot be debugged from the same computer.

- Additional improvements to the debugger interface include the following:
 - ❑ Support for browsing memory in the debugger. The Memory Viewer lets you view the contents of memory when execution of a process being debugged is paused. You can step through memory or specify addresses, variable names and some expressions.
 - ❑ Support for viewing global variables in the debugger. The Xcode debugger now shows the values of global variables in the variables list of the debugger window. Using the globals browser, you can also browse for global variables by library and add them to the debugger's variables list.
- Subversion support. Xcode adds support for the Subversion source code management system. Xcode already supports CVS and Perforce.
- Numerous user interface enhancements designed to give you better feedback and improve your workflow. Xcode introduces the following features:
 - ❑ There is a new activity viewer, which lets you view the progress of all tasks running in Xcode, across all open projects.
 - ❑ Inspector interfaces have been improved, with easier editing of lists of strings in the build settings view. There is also an inspector interface for editing executable environments.
 - ❑ Most Xcode windows, such as the Build Results and Find Results windows, now include embedded editors, for viewing and modifying source in the same window.

For more information about features and improvements of Xcode, see the Xcode Release Notes. In Xcode, you can view these notes by choosing Help > Show Release Notes.

Access Control Lists

Mac OS X "Tiger" introduces support for Access Control Lists (ACLs)—a robust system for implementing file-based permissions. ACLs offers many improvements over the existing BSD permissions currently used by the Mac OS X file systems. Among these improvements are the following:

- Support for ownership of files and directories by a group
- Support for multiple owners of a file or directory, each with potentially different permissions
- Enhanced interoperability with Samba and Windows
- More control over a file than just read/write/execute permissions
- Support for static inheritance of file permissions from a parent directory

ACLs are a feature widely used in enterprise computing because it provides flexible and highly configurable rights management for servers. It removes many of the limitations of the existing BSD permissions by allowing access to files and directories by multiple groups and users. In addition, it lets the system administrator grant specific rights to each user and group without requiring the creation of special new groups. It also expands the rights that can be granted beyond read, write, and execute permissions to include the ability to delete files or read a file's attributes.

For many software developers, ACLs may have little impact on the code you write; you can continue to use the same APIs to create or open files and read their contents. If you are developing a program that interacts more closely with the file system, such as an archiving or backup utility, you need to be aware of ACLs and able to handle them. Similarly, if your program involves collaboration among users over a network (or on the same computer), you might need to use ACLs to ensure each user has the appropriate access to the files being shared.

Apple provides an ACL scheme for HFS+ volumes and supports the ACL schemes promoted by other network file systems. Support for getting and setting access control entry (ACE) records on a file or directory is implemented in the POSIX layer of the system. Although ACLs are always enabled on Tiger, the system does not automatically map existing permissions to ACE records. Instead, programs that want to use ACLs must create and manage those entries using the POSIX interfaces.

See the `unistd.h` and `acl.h` header files for the list of functions you can use to manage ACLs.

Networking Enhancements

Mac OS X "Tiger" introduces several networking-related improvements. The following sections describe these improvements and offer advice on how you can take advantage of them in your software.

Rendezvous Enhancements

Tiger introduces enhancements to Rendezvous that allow it to support browsing across subnets. Whereas previous versions of Rendezvous returned only the local subnet (`.local`) when asked for a list of available domain names, the new version returns the list of all available domain names.

If you have code that assumes there is only one domain, you should modify your code to handle the additional information. You should also check any code that receives a domain name as part of a callback to make sure it does not simply assume that the `.local` domain was returned. If you do not specify a domain when browsing for services, Rendezvous automatically searches the local domain and the user's .Mac account for available services.

If your program provides a dialog or window that lets the user select Rendezvous-based services, you need to reconsider the way you display that information. In addition to choosing a service, you may want users to be able to choose a domain, which adds another hierarchical level to the selection process. If that's the case, you need to provide additional controls and logic for selecting a domain and then selecting the services in that domain. The Tiger developer tools include sample code for creating hierarchical dialogs. See `/Developer/Examples` for more information.

CFNetwork Enhancements

CFNetwork includes the following enhancements:

- Improved interoperability with NT LAN Manager
- Performance improvements
- Improved security

CFNetwork provides abstractions to make network-based communications easier. It is designed for programs that work with BSD sockets, manage information about remote hosts, or interact with HTTP, HTTPS, and FTP servers. If your program performs any of these tasks and does not currently use CFNetwork, you should investigate this technology.

For more information about CFNetwork, see *CFNetwork Services* in Networking Core Foundation Documentation.

Network Diagnostics

Although modern networks are generally reliable, there are still times when network services may fail. Sometimes the cause of the failure is beyond the ability of the desktop user to fix, but sometimes the problem is in the way the user's computer is configured. The network diagnostics feature of Tiger provides a diagnostic application to help the user locate problems and correct them.

If your application encounters a network error, you can use the new diagnostic interfaces of CFNetwork to launch the diagnostic application and attempt to solve the problem interactively. You can also choose to report diagnostic problems to the user without attempting to solve them.

For more information on using this feature, see the header files of CFNetwork.

KEXT Stable API

The only way to implement some low-level features of Mac OS X "Tiger," such as a network stack or new file system, is through a kernel extension (also known as a **KEXT**). The process for creating a KEXT is different from the one for creating a device driver. Device driver interactions occur through the I/O Kit, which isolates the driver code from kernel details and has provided a stable base of operations for some time. True kernel extensions used to interact directly with the data structures of the kernel environment, which lead to problems when those data structures changed.

Starting with Mac OS X "Tiger", the data structures of the kernel environment are migrating toward a more opaque access model to prevent changes from affecting developers in the future. Instead of accessing kernel structures directly, clients will now need to use accessor functions defined by the kernel environment. There will be a transition period to give developers time to migrate their code to the new accessor-based model. This transition is required for developers of network kernel extensions and file systems and will alleviate many of the previous problems caused by changes to the kernel environment.

Important

The new API for accessing kernel data structures is not necessary for I/O Kit developers. The I/O Kit interfaces are unaffected by these changes.

For information about the interfaces of the new API, see the header files or the available HeaderDoc documentation.

Document Revision History

The table below describes the revisions to *Tiger Technology Overview*.

Date	Notes
June 30, 2004	First version of <i>Tiger Technology Overview</i> .

REVISION HISTORY

Document Revision History